

Campus Gaffer

Technical Design Document

Data Acquisition & Pipeline Architecture

Version 1.0 • March 2026 • Pre-Build Reference

Status Pre-Build Draft	Audience Solo Developer	Phase MVP — Intramural Soccer
----------------------------------	-----------------------------------	---

1. Purpose of This Document

This document serves as the single source of truth for every architectural decision made before writing production code. It is intentionally prescriptive — each section answers a question your future self will ask at 2 am when something breaks.

WHY

Your architecture summary contained a working script and a list of steps but no explicit decisions. A design document forces you to answer the hard questions before they become hard bugs.

Specifically, this document answers:

- What is this system, in one sentence?
- What does it NOT do? (scope boundaries)
- What are the domain entities, and what are their identities?
- Where are the layer boundaries, and what crosses them?
- What is the failure model? (what breaks, how loud is it)
- How do we handle the external dependency (IMLeagues) going away?

2. Problem Statement & Goals

2.1 What We Are Building

Campus Gaffer is a fantasy sports platform for university intramural leagues. The MVP is a soccer fantasy game modelled on the Premier League Fantasy experience. The backend has two responsibilities:

1. Acquire game and player data automatically from IMLeagues.
2. Expose that data via a clean API to the frontend.

FRONTEND NOTE

The frontend is mobile-first and web-based — not React Native. This is a deliberate scoping decision: a responsive web app is faster to ship from zero, easier to iterate on, and avoids app store friction. Native wrappers (e.g. Capacitor) can be layered on later if needed. The backend API contract is identical either way.

NORTH STAR

The long-term goal is platform independence: Campus Gaffer owns its own data. IMLeagues is a temporary data source, not a permanent dependency. Every design decision should make it easier to swap out IMLeagues for another source — or no source at all.

2.2 What We Are NOT Building (MVP Scope Boundary)

Explicitly out of scope for v1.0:

- Any sport other than soccer
- Real-time score streaming (polling every ~60 min is acceptable)

- Headless browser / automated cookie renewal (manual admin override only)
- Multi-tenant / multi-university support
- Any direct write-back to IMLeagues

DECISION

Defining scope is a decision. Leaving scope implicit is how projects bloat. If a stakeholder asks for basketball in week 2, the answer is: it is not in v1.0 scope — point to this document.

3. Architecture Overview

3.1 System Layers

The system is divided into four horizontal layers. Data flows strictly downward. No layer talks to a layer above it.

Layer	Package	Responsibility
Infrastructure	AWS (EventBridge, Lambda, ECS, SSM, CloudWatch)	Scheduling, compute, secrets, and observability. Owned by AWS — not application code.
External	—	IMLeagues API. Treat as hostile. May vanish without notice.
Scraper	internal/scraper	HTTP + parsing only. Returns scraper-native DTOs. Zero knowledge of DB.
Service	internal/service	Orchestration, translation, upsert logic. The only layer that knows both DTOs and models.
Repository	internal/repository	GORM + Neon PostgreSQL. Accepts domain models only. No scraper types enter here.
API	internal/api	HTTP handlers. Reads from repository. Returns JSON to frontend. No scraper logic.

DECISION

The service layer is the translation boundary. ScrapedGame (scraper DTO) never appears in a GORM model. Game (domain model) never appears in a scraper function. This is not over-engineering — it is the mechanism that allows you to swap IMLeagues out later.

**INFRASTRUCTURE
NOTE**

The scraper runs as an AWS Lambda function invoked by an EventBridge cron rule. The API runs as an ECS Fargate Service behind an Application Load Balancer. These are separate compute units — not co-located on a single server. A pipeline failure cannot affect API availability.

3.2 Data Flow — Happy Path

The following describes a single end-to-end pipeline run. This is the canonical sequence every engineer should be able to recite from memory.

```

Cron fires (e.g. every 60 min)
↓
[Scraper] DiscoveryScraper.GetCurrentSeasonGames()
        returns []ScrapedGame (scraper-native DTO, no DB types)
↓
[Service] GameService.SyncGames([]ScrapedGame)
        for each ScrapedGame:
            • check DB: does external_game_id already exist?
            • if not → INSERT with is_scraped = false
            • if yes → skip (idempotent)
↓
[Service] GameService.ScrapeUnscrapedGames()
        fetches all Games WHERE status='Completed' AND is_scraped=false
        for each → calls StatsScraper.GetGameStats(externalID)
            → translates ScrapedStats → PlayerPerformance
            → upserts via repository
            → sets is_scraped = true
↓
[Repository] All writes committed. Pipeline complete.

```

4. Domain Model & Identity Strategy

4.1 The Identity Problem

This is the most consequential design decision in the project. Get it wrong and you are coupled to IMLeagues forever. Get it right and replacing IMLeagues is a 1-day job.

The wrong approach — what most scrapers do:

```

// BAD: IMLeagues ID is the primary key
type Game struct {
    IMLeaguesGameID string `gorm:"primaryKey"`
    // ...
}
// Problem: Your entire foreign key graph now speaks IMLeagues.
// If IMLeagues IDs change, or you add a second source,
// you have to migrate EVERYTHING.

```

The correct approach — surrogate key + natural key pattern:

```

// GOOD: Internal UUID is primary key. IMLeagues ID is metadata.
type Game struct {
    ID uuid.UUID
    `gorm:"primaryKey;type:uuid;default:gen_random_uuid()"`
    ExternalGameID string
    `gorm:"uniqueIndex:idx_game_external;column:external_game_id"`
    ExternalSource string `gorm:"uniqueIndex:idx_game_external" //
    "imleagues"
    Status string // "Scheduled" | "Completed" | "Cancelled" - load-
bearing for FindUnscraped
    IsScraped bool `gorm:"default:false"`
}

```

```

    CreatedAt      time.Time
    UpdatedAt      time.Time // updated by upsert on every status change
}
// The composite unique index (ExternalGameID, ExternalSource) enforces
// uniqueness per source while keeping the primary key internal.

```

DECISION

The rest of the system — foreign keys, API responses, fantasy scoring — refers to Game.ID (UUID). Game.ExternalGameID is used only in the scraper layer to look up whether we have already seen this game. The word "imleagues" appears in exactly two places: ExternalSource values and the scraper package.

4.2 Core Domain Entities

Entity	Key Fields	Notes
Game	id, external_game_id, external_source, status, is_scraped, updated_at	Status is load-bearing: FindUnscraped queries WHERE status='Completed' AND is_scraped=false
Team	id, external_id, external_source, name, league_id	A team exists independently of any single game.
Player	id, external_player_id, external_source, name	One player across many seasons and teams.
PlayerPerformance	id, player_id, game_id, goals, game_played, is_mvp	The grain: one row per player per game.
League / Season	id, external_id, name, sport, year	Top-level container. All games belong to a season.

5. Scraper Layer Design

5.1 The IMLeagues API Contract

IMLeagues is an ASP.NET SPA. Its data endpoint follows a deterministic command pattern:

```

POST https://www.imleagues.com/Services/AjaxRequestHandler.ashx
Content-Type: application/x-www-form-urlencoded

jsonData={
  "class":      "imLeagues.Web.Members.Services.BO.<Namespace>.ViewGameBO",
  "method":     "Initialize",
  "paramType":  "imLeagues.Internal.API.VO.Input.<Namespace>.ViewGameInVO",
  ... entity params ...
}

```

The three scraper targets for MVP, in priority order:

Target	Suspected Endpoint Class	Output
Game schedule	BO.League.TeamHomeBO (mine existing captures first)	List of gameId, status, date
Game stats	BO.League.ViewGameBO (already working)	Attendance + HTML-embedded goal data
Team roster	BO.League.TeamRosterBO (TBD)	Player name, memberId

TACTIC

Before opening the browser Network tab to hunt new endpoints, first mine the team_page.json you already have. The fields gameIdWithType and gameResultStr in that response likely give you a full schedule. Crawl what you own before you scrape what you don't.

5.2 Scraper-Native DTOs

Scraper functions return types that live only in the scraper package. They are not GORM models. They are not API responses. They are raw data envelopes.

```
// internal/scraper/dto.go

type ScrapedGame struct {
    ExternalGameID string
    ExternalSource string // always "imleagues"
    HomeTeamName   string
    AwayTeamName   string
    GameResultStr  string // "W" | "L" | "D" from team perspective - NOT a
    completion signal
    GameResultScore string // non-empty string = game has been played; empty =
    not yet played
    ScheduledAt    time.Time
}

type ScrapedGameStats struct {
    ExternalGameID string
    HomeStats      []ScrapedPlayerStat
    AwayStats      []ScrapedPlayerStat
}

type ScrapedPlayerStat struct {
    ExternalPlayerID string
    PlayerName       string
    GamePlayed       bool
    IsMVP            bool
    Goals            int
}
```

RULE

A scraper function's return type must never import from internal/models or internal/repository. If you find yourself importing a GORM model in a scraper file, you have crossed a layer boundary. Stop and refactor.

5.3 Cookie Authentication & Failure Handling

Cookie expiration is the primary reliability risk in this system. The design principle is: fail loudly, never silently.

Scenario	HTTP Signal	Required Behaviour
Cookie valid	200 OK + isDone:true	Parse and proceed normally.
Cookie expired	200 OK + HTML login redirect	Detect redirect in body. Return typed error: ErrSessionExpired. Log + alert. DO NOT silently return empty results.
Hard 401/403	401 or 403	Return typed error: ErrUnauthorized. Same alert path.
Rate limited	429 or 503	Exponential backoff $\times 3$, then return ErrRateLimited.

Cookie storage and rotation — how it works in production:

- The cookie string lives in AWS SSM Parameter Store as a SecureString. It is never in source code, env vars, or server memory.
- Lambda reads the cookie from SSM at the start of each invocation — not at deploy time. Cookie rotation takes effect on the next run with zero redeployment.
- The admin POST /admin/update-cookie endpoint writes the new value directly to SSM. The next Lambda invocation picks it up automatically.
- ErrSessionExpired and ErrUnauthorized log to CloudWatch Logs. A CloudWatch Metric Filter on the log group feeds a CloudWatch Alarm, which publishes to an SNS Topic (email or SMS) to the admin.

```
// Lambda handler reads cookie at invocation time — not at startup
func handler(ctx context.Context, event events.CloudWatchEvent) error {
    ssmClient := ssm.NewFromConfig(cfg)
    param, err := ssmClient.GetParameter(ctx, &ssm.GetParameterInput{
        Name:          aws.String("/campus-gaffer/imleagues-cookie"),
        WithDecryption: aws.Bool(true),
    })
    if err != nil {
        return fmt.Errorf("failed to read cookie from SSM: %w", err)
    }
    cookie := *param.Parameter.Value
    // ... pass cookie into scraper ...
}
```

6. Service Layer Design

6.1 GameService — The Orchestrator

The service layer is where the pipeline lives. It is the only code that touches both scraper DTOs and domain models. It contains all business logic.

```
// internal/service/game_service.go
```

```
type GameService struct {
    discovery scraper.DiscoveryScraper
    stats      scraper.StatsScraper
    gameRepo   repository.GameRepository
    playerRepo repository.PlayerRepository
    perfRepo   repository.PerformanceRepository
}

// SyncGames: Phase 1 + 2
// Takes scraped games, upserts new ones, marks known ones.
func (s *GameService) SyncGames(ctx context.Context, scraped []scraper.ScrapedGame) error

// ProcessCompletedGames: Phase 3 + 4
// Finds all Completed + !is_scraped games, fetches stats, persists.
func (s *GameService) ProcessCompletedGames(ctx context.Context) error
```

DECISION

SyncGames and ProcessCompletedGames are separate methods, not one mega-function. This means you can test discovery without touching the stats scraper, and vice versa. It also means a stats-scraper failure does not roll back the discovery phase.

6.2 Translation: ScrapedGame → Game

Translation is a pure function. It has no side effects and is trivially testable.

```
func toGame(s scraper.ScrapedGame) models.Game {
    return models.Game{
        // ID is generated by the DB — do NOT set it here
        ExternalGameID: s.ExternalGameID,
        ExternalSource: s.ExternalSource, // "imleagues"
        Status:         normaliseStatus(s.GameResultScore),
        IsScraped:      false,
        ScheduledAt:    s.ScheduledAt,
    }
}

// normaliseStatus maps IMLeagues fields to our domain enum.
// Signal: GameResultScore — non-empty means the game has been played.
// GameResultStr carries W/L/D from the team's perspective and must NOT
// be used as a completion signal — it is non-empty for all finished games
// but gives no useful status information on its own.
func normaliseStatus(gameResultScore string) string {
    if gameResultScore != "" {
        return "Completed"
    }
    return "Scheduled"
}
```

7. Repository Layer Design

7.1 Upsert Pattern

All writes are upserts, not blind inserts. This is what makes the pipeline idempotent — you can run it ten times and the DB stays consistent.


```
// Postgres upsert via GORM - GameRepository
func (r *gameRepo) Upsert(ctx context.Context, g *models.Game) error {
    return r.db.WithContext(ctx).
        Clauses(clause.OnConflict{
            Columns: []clause.Column{{Name: "external_game_id"}, {Name:
"external_source"}},
            DoUpdates: clause.AssignmentColumns([]string{"status", "updated_at"}),
        }).
        Create(g).Error
}
// If (external_game_id, external_source) already exists → update status only.
// is_scraped is intentionally NOT in DoUpdates - once true, it stays true.
```

RULE

`is_scraped` is a one-way flag. Once set to true, no upsert path ever sets it back to false. To reprocess a game, an admin must explicitly reset it. This prevents accidental double-processing.

7.2 Repository Interfaces

Each repository is defined as a Go interface. The GORM implementation satisfies the interface. This gives you testable code without a real database.

```
type GameRepository interface {
    Upsert(ctx context.Context, g *models.Game) error
    FindByExternalID(ctx context.Context, src, id string) (*models.Game, error)
    FindUnscraped(ctx context.Context) ([]models.Game, error)
    MarkScraped(ctx context.Context, id uuid.UUID) error
}
```

8. Pipeline & Scheduling

8.1 The Four Phases

#	Phase	Code Location	Exit Condition
1	Discovery	scraper.DiscoveryScraper.GetCurrentSeasonGames()	Returns []ScrapedGame or typed error
2	State Sync	service.GameService.SyncGames()	All new games inserted; known games skipped
3	Stats Fetch	scraper.StatsScraper.GetGameStats(externalID)	Returns ScrapedGameStats per completed game
4	Persistence	service.GameService.ProcessCompletedGames()	<code>is_scraped</code> flipped to true; perf rows inserted

8.2 Cron Job Structure

Because the intramural league runs one game per week, there is no need for an hourly polling cycle. The cron is scheduled to run on weekends — this is when school admins are most likely to have finalised scores and updated IMLeagues. Two weekend runs (Saturday morning and Sunday morning) are used for resilience: if Saturday's run catches nothing yet, Sunday's will.

DECISION

The cron is not managed in application code. The robfig/cron scheduler is removed. Scheduling is owned by an EventBridge Rule. The Lambda handler replaces the anonymous cron function — it receives the EventBridge event, runs the four pipeline phases, and returns. The application has no concept of time or scheduling internally. Scheduling is an infrastructure concern, not an application concern.

```
# EventBridge Rule — runs at 09:00 UTC every Saturday and Sunday
# cron(Minutes Hours Day-of-month Month Day-of-week Year)
cron(0 9 ? * SAT,SUN *)

# Lambda handler signature — no scheduler in application code
func handler(ctx context.Context, event events.CloudWatchEvent) error {
    // Phase 1 + 2: discover and sync games
    games, err := discoveryScraper.GetCurrentSeasonGames(ctx)
    if err != nil {
        log.Printf("ERROR ErrSessionExpired: %v", err) // triggers CloudWatch
        Alarm
        return err
    }
    if err := gameService.SyncGames(ctx, games); err != nil {
        log.Printf("ERROR pipeline sync failed: %v", err)
        return err
    }

    time.Sleep(5 * time.Second) // rate-limit courtesy sleep

    // Phase 3 + 4: fetch stats and persist
    if err := gameService.ProcessCompletedGames(ctx); err != nil {
        log.Printf("ERROR stats processing failed: %v", err)
        return err
    }
    return nil
}
```

RULE

Every Lambda invocation is independent. A failure in one run must not poison the next. Because the pipeline is idempotent and `is_scraped` is a one-way flag, simply letting the next scheduled invocation fire is the recovery mechanism for most failure modes. The weekend-only schedule also dramatically reduces the risk of IP-based rate limiting — you are making roughly 2 requests per week, not 168.

9. Error Taxonomy & Failure Model

Classify every error before writing the handler. This prevents the worst failure mode: a silent empty result that looks like success.

All `log.Printf` output lands in CloudWatch Logs automatically — no changes to logging code required. Errors marked with a CloudWatch Alarm trigger an SNS notification (email or SMS) to the admin.

Error Type	Typed Sentinel	Recoverable?	Action	Alert Destination
Session expired	ErrSessionExpired	Yes (manual)	Log + alert admin. Halt run. Do NOT return empty slice.	CloudWatch Alarm → SNS (email/SMS)
Hard auth failure	ErrUnauthorized	Yes (manual)	Same as above.	CloudWatch Alarm → SNS (email/SMS)
Rate limited	ErrRateLimited	Yes (auto)	Exponential backoff × 3, then alert.	CloudWatch Logs
Parse failure	ErrParseFailed	Partial	Log offending game ID + raw payload. Skip that game. Continue pipeline.	CloudWatch Logs
DB write failure	ErrDBWrite	Yes (auto)	Retry × 3 with backoff. Alert if all retries fail.	CloudWatch Logs
Network timeout	ErrTimeout	Yes (auto)	Retry × 2. Alert on persistent failure.	CloudWatch Logs

CRITICAL

The most dangerous failure is returning an empty `[]ScrapedGame` when the cookie has expired. From the pipeline's perspective this looks identical to "no new games." You would silently stop persisting results and not know why. Detect this explicitly.

10. Open Questions & Decision Log

10.1 Decisions Made (Locked)

Decision	Rationale
Surrogate UUID primary keys	Decouples identity from IMLeagues. All FK relationships use internal IDs.
external_game_id + external_source composite index	Supports multiple data sources without schema changes. Column naming is entity-specific (external_game_id, external_player_id) for schema clarity.
Scraper DTOs separate from GORM models	Layer separation: scraper layer can be replaced without touching the DB schema.
Service layer owns translation	Single responsibility. Scraper = fetch. Service = orchestrate. Repo = persist.
Idempotent upserts everywhere	Pipeline runs are naturally retried. Idempotency is free correctness.
is_scraped is one-way	Prevents double-processing without complex distributed locking.
Fail loudly on session expiry	Silent empty results are worse than noisy failures.

Decision	Rationale
Scraper runs as AWS Lambda	Zero idle cost; perfect fit for a run-and-done job that executes ~30s twice a week. No server to babysit.
API runs as ECS Fargate Service	Long-lived compute that scales. Separate from the scraper so a pipeline failure cannot affect API availability.
Cookie stored in SSM Parameter Store (SecureString)	Auditable, IAM-controlled, no plaintext in env vars or code. Lambda reads at invocation time — cookie rotation takes effect without redeployment.
Scheduling owned by EventBridge, not application code	Separation of concerns. The application should not know what day it is.
Alerts via CloudWatch Alarm → SNS	Closes the alert delivery question. Keeps observability entirely within AWS — no third-party dependency for MVP.

10.2 Open Questions (Must Resolve Before Production)

Question	Impact If Deferred
What is the exact discovery endpoint? (TeamHomeBO vs. a schedule-specific BO?)	Phase 1 cannot be built until this is known. Mine existing team_page.json captures first.
Does gameIdWithType encode gameType? Is the format stable?	GetGameStats call signature depends on this.
What does a cancelled/postponed game look like in GameResultScore?	If GameResultScore is non-empty for a cancelled game, normaliseStatus will misclassify it as Completed. Verify against a real cancelled fixture before deploying.
What IAM permissions does the Lambda execution role require?	Minimum: ssm:GetParameter on the cookie parameter, secretsmanager if Neon credentials are stored there, and logs:CreateLogGroup / logs:PutLogEvents. Must be locked before first deploy.
Will the Neon DB free tier sustain the write volume at MVP?	~2 games/week x 22 players x 30 weeks = ~1,320 perf rows/season. Likely fine, but verify.
What is the pricing model shape? (static per season vs. dynamic week-to-week)	Static = single price column on Player. Dynamic = separate price history table. This determines whether the Player schema needs revision before the repository layer is built. Must be decided before API layer work begins.

11. Implementation Roadmap

Ordered by dependency. Do not start a step until the one before it has passing tests.

3. Define all DTOs and domain models (no logic, just types). This is the contract everything else depends on.

4. Implement repository interfaces + GORM implementations. Write integration tests against a local Postgres / Neon dev branch.
5. Wire the existing ViewGameBO scraper to return ScrapedGameStats (the DTO, not models.PlayerPerformance).
6. Implement the translation functions in the service layer. Unit-test them with known fixtures.
7. Implement GameService.SyncGames + GameService.ProcessCompletedGames end-to-end.
8. Discover the schedule endpoint. Implement DiscoveryScraper.GetCurrentSeasonGames.
9. Wire up the Lambda handler entry point and deploy with EventBridge Rule (replaces cmd/scraper/main.go cron daemon).
10. Implement the admin cookie-update endpoint (writes to SSM) + CloudWatch Alarm / SNS alerting.
11. Load test the pipeline against a full season of fixtures.

REMINDER

Each step produces a testable artefact before moving on. The order is: types → persistence → scraping → orchestration → scheduling. Never the reverse.

12. Scoring Inputs & PlayerPerformance Contract

This section locks the data that the pipeline must persist per player per game. It does not define the scoring algorithm — that is post-MVP. It defines the inputs that algorithm will consume, which is an architecture concern: you cannot build the repository layer without knowing what PlayerPerformance must hold.

RULE

The scraper is the ceiling. If IMLeagues does not expose a stat, it cannot be in PlayerPerformance, and it cannot be in the scoring algorithm. Do not design a scoring system that requires data the pipeline cannot produce.

12.1 What IMLeagues Exposes Per Player Per Game

These are the three fields the ViewGameBO endpoint returns at the player level. They are the complete input set available to the MVP scoring algorithm — nothing more can be added without a new data source.

Field	Source	Notes
goals	team1/2PlayerStatsUC (HTML)	Parsed from embedded HTML table via goquery. Integer.
game_played	team1/2MemberAttendanceList	markedPlay boolean from attendance JSON. Whether the player took the field.
is_mvp	team1/2MemberAttendanceList	isMVP boolean from attendance JSON. At most one per team per game.

12.2 PlayerPerformance — Locked Schema

These are the fields that must be present on PlayerPerformance. The schema is closed for MVP — no additional stat columns should be added until a second data source is available.

```
// internal/models/player_performance.go

type PlayerPerformance struct {
    ID          uuid.UUID `gorm:"primaryKey;type:uuid;default:gen_random_uuid()"`
    PlayerID    uuid.UUID `gorm:"not null;index"`           // FK → Player.ID
    GameID      uuid.UUID `gorm:"not null;index"`           // FK → Game.ID
    Goals       int       `gorm:"not null;default:0"`
    GamePlayed  bool      `gorm:"not null;default:false"`
    IsMVP       bool      `gorm:"not null;default:false"`
    CreatedAt   time.Time
    UpdatedAt   time.Time
}
// Grain: one row per player per game.
// Unique constraint on (player_id, game_id) — a player cannot have
// two performance records for the same game.
```

NOTE

GamePlayed being false means the player was on the roster but did not take the field. These rows are still persisted — a player being absent is data, not a gap. The scoring algorithm decides whether absence earns zero points or is simply excluded from ranking.

13. Glossary

Term	Definition
ScrapedGame	Scraper-layer DTO. Lives in internal/scraper. Contains raw IMLeagues field values.
Game (model)	Domain model. Lives in internal/models. Uses internal UUID. Never references IMLeagues directly.
external_game_id	The game ID as IMLeagues knows it. Metadata, not identity. Stored on the Game model; used as the conflict column in upserts.
external_source	The string 'imleagues'. Part of the composite unique index. Enables multi-source future.
is_scraped	Boolean flag on Game. True = stats have been fetched and persisted. One-way.
gameResultStr	Raw IMLeagues field. Carries W/L/D from the team's perspective. NOT a completion signal — do not use to detect whether a game has been played.
gameResultScore	Raw IMLeagues field. Non-empty = game has been played. Empty = not yet played. This is the correct completion signal for normaliseStatus.
Discovery Scraper	Fetches the schedule. Returns a list of games and their statuses.
Stats Scraper	Fetches per-game attendance and goal data given a game external ID.
Upsert	Insert-or-update. On conflict (external_game_id, external_source): update status and updated_at only.
ErrSessionExpired	Typed sentinel error returned when cookie-based auth has lapsed.